OCR API: Deployment Quick Guide (Ubuntu)

Ubuntu 22.04 / 24.04 · Python 3.11 · uvicorn + nginx · URL: http://192.168.6.128/ocr_api/docs

Run everything as root on the Linux server unless the step says Windows. On Ubuntu, open a root shell first with:

```
sudo -i
```

Item	Value used in this guide
Server IP	`192.168.6.128`
Project folder	`/opt/ocr_api/ocr_api` (contains the `app/` folder)
Virtual env	`/opt/ocr_api/ocr_api/mvenv`
App entry point	`app.main:app`
Public URL prefix	`/ocr_api`
Ubuntu login user	`youruser` (replace with yours; used for FileZilla)

VM sizing (VMware): at least 8 GB RAM (16 GB recommended) and at least 40 GB disk. Smaller VMs fail during pip install with “Killed” (out of memory) or “No space left on device”.

What changes from Rocky: `apt` instead of `dnf`; Python 3.11 from the deadsnakes PPA; no SELinux (no `setsebool`); `ufw` instead of `firewall-cmd`; nginx site in `sites-available` and the default site disabled by removing a link; PostgreSQL needs no `--initdb`; services are `redis-server` and `ssh`; root cannot log in over SSH, so FileZilla uses your own user.

1. Install system packages

```
apt update
apt install -y software-properties-common
add-apt-repository -y ppa:deadsnakes/ppa
apt update

apt install -y python3.11 python3.11-venv python3.11-dev \
  build-essential pkg-config unzip \
  libkrb5-dev krb5-user \
  tesseract-ocr tesseract-ocr-eng tesseract-ocr-ara \
  libgl1 libglib2.0-0 libgomp1 \
  postgresql redis-server nginx openssh-server
```

2. Add swap (protects pip and the app from running out of memory)

Ubuntu may already have a swap file (`/swap.img`). Check with `swapon --show`; add one if there is none or it is small:

```
fallocate -l 4G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab
free -h
```

3. Start PostgreSQL and Redis

Ubuntu initialises PostgreSQL during install, so there is no `postgresql-setup --initdb`.

```
systemctl enable --now postgresql redis-server
```

Create the database and user (replace the names and password with yours):

```
sudo -u postgres psql -c "CREATE USER ocr_user WITH PASSWORD 'ChangeMe123';"
sudo -u postgres psql -c "CREATE DATABASE ocr_db OWNER ocr_user;"
```

4. Copy the project from Windows (FileZilla)

1. Make sure SSH is running on the server and create the project folder:

```
systemctl enable --now ssh
mkdir -p /opt/ocr_api
```

2. In FileZilla, connect with Host `sftp://192.168.6.128`, your Ubuntu user `youruser` (not root; Ubuntu blocks root SSH login), port 22.

3. Upload a zip of the project without `.venv`, `venv`, `__pycache__` and `.git` to your home folder `/home/youruser/`.

4. Move it into place and unzip it on the server:

```
mv /home/youruser/ocr_api.zip /opt/ocr_api/
cd /opt/ocr_api
unzip -o ocr_api.zip -d /opt/ocr_api/ocr_api
ls /opt/ocr_api/ocr_api        # must show app/ and the requirements files
```

Also upload the QARI/Qwen2-VL model weights, and the Windows folders `C:\Users\<you>\.EasyOCR` and `C:\Users\<you>\.paddlex` to `/home/youruser/`, then move the two model folders to `/root/` (the service runs as root). The app runs offline and won't download models.

```
mv /home/youruser/.EasyOCR /home/youruser/.paddlex /root/
```

> Never put the project in `/var/www/html`. That is nginx's public folder on Ubuntu, so your code and secrets would be downloadable.

5. Create the virtual environment with Python 3.11

```
cd /opt/ocr_api/ocr_api
python3.11 -m venv mvenv
source mvenv/bin/activate
python --version               # must show Python 3.11.x
pip install --upgrade pip wheel
pip install --no-cache-dir -r base.txt -r cpu.txt
```

Check the install:

```
python - <<'PY'
import torch, cv2, paddle, easyocr, transformers, gssapi, pytesseract
print('torch', torch.__version__)
print('tesseract', pytesseract.get_tesseract_version())
PY
```

> Activate with `source mvenv/bin/activate`, not `mvenv/bin/activate` and not with `sudo`. Always create venvs with `python3.11`. Never change the system `python3`, because Ubuntu's apt tools depend on it.

6. Update the app configuration (.env)

- Set the environment to production.
- Database URL: `postgresql://ocr_user:ChangeMe123@localhost:5432/ocr_db`
- Redis URL: `redis://localhost:6379/0`
- Replace all Windows paths (`C:\...`) with Linux paths. Tesseract is `/usr/bin/tesseract`.
- Point the model folders to where you uploaded the weights.

7. Test run

```
cd /opt/ocr_api/ocr_api
source mvenv/bin/activate
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Wait for `Application startup complete`, then stop it with Ctrl+C.

8. Run the app as a systemd service

```
cat > /etc/systemd/system/ocr-api.service <<'EOF'
[Unit]
Description=OCR API
After=network.target postgresql.service redis-server.service

[Service]
WorkingDirectory=/opt/ocr_api/ocr_api
ExecStart=/opt/ocr_api/ocr_api/mvenv/bin/uvicorn app.main:app \
    --host 127.0.0.1 --port 8000 --root-path /ocr_api
Restart=always
User=root

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now ocr-api
systemctl status ocr-api --no-pager
curl -I http://127.0.0.1:8000/docs          # expect HTTP 200
```

9. Configure nginx

a) Disable the default site. On Ubuntu this is a link, not a block inside `nginx.conf`, so don't edit `nginx.conf`:

```
rm -f /etc/nginx/sites-enabled/default
```

b) Create the site config and enable it:

```
cat > /etc/nginx/sites-available/ocr_api <<'EOF'
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;

    client_max_body_size 50M;

    location = /ocr_api {
        return 301 /ocr_api/;
    }

    location /ocr_api/ {
        proxy_pass http://127.0.0.1:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Prefix /ocr_api;
        proxy_read_timeout 300s;
    }
}
EOF

ln -s /etc/nginx/sites-available/ocr_api /etc/nginx/sites-enabled/ocr_api
```

10. Open the firewall, start nginx

No SELinux on Ubuntu, so `setsebool -P httpd_can_network_connect 1` is not needed. Check the firewall:

```
ufw status
```

If it says **inactive**, skip to the nginx commands. If it is **active**:

```
ufw allow 'Nginx HTTP'
ufw reload
```

> If you turn ufw on (`ufw enable`), run `ufw allow OpenSSH` first, or you lose SSH and FileZilla access.

```
nginx -t
systemctl enable --now nginx
systemctl reload nginx
```

11. Open the app

From Windows, open `http://192.168.6.128/ocr_api/docs` and press Ctrl+F5.

After updating code

```
# upload the new files with FileZilla to /home/youruser,
# unzip into /opt/ocr_api/ocr_api, then:
systemctl restart ocr-api
```

If requirements changed:

```
cd /opt/ocr_api/ocr_api && source mvenv/bin/activate
pip install --no-cache-dir -r base.txt -r cpu.txt
systemctl restart ocr-api
```

Useful commands

Task	Command
App status	`systemctl status ocr-api`
App logs (live)	`journalctl -u ocr-api -f`
Restart app	`systemctl restart ocr-api`
nginx config test	`nginx -t`
Reload nginx	`systemctl reload nginx`
nginx error log	`tail -f /var/log/nginx/error.log`
Enabled nginx sites	`ls -l /etc/nginx/sites-enabled/`
Firewall status	`ufw status`
Memory / disk	`free -h / df -h`

Troubleshooting

Error	Fix
FileZilla: Access denied for root	Normal on Ubuntu. Log in as your own user and upload to `/home/youruser`.
FileZilla: Connection refused	`systemctl enable --now ssh`; if ufw is active, `ufw allow OpenSSH`.
activate: Permission denied	Use `source mvenv/bin/activate`.
No matching distribution found for peft==0.21.0	The venv uses the wrong Python (3.10 / 3.12). Delete it and recreate with `python3.11 -m venv mvenv`.
python3.11: command not found	Add the deadsnakes PPA and install `python3.11 python3.11-venv python3.11-dev` (step 1).
krb5-config: command not found	`apt install -y libkrb5-dev build-essential python3.11-dev`
Killed / Out of memory during pip	Add swap (step 2), use `--no-cache-dir`, increase VM RAM.
No space left on device	`pip cache purge`; `apt clean`, then expand the VM disk.
libGL.so.1: cannot open shared object file	`apt install -y libgl1 libglib2.0-0 libgomp1`
Other missing lib….so	`apt install -y apt-file && apt-file update`, then `apt-file search libNAME.so` and install the package it names.
“Welcome to nginx!” or nginx 404 page	The default site is still enabled. `rm -f /etc/nginx/sites-enabled/default` (step 9a), then `systemctl reload nginx`.
502 Bad Gateway	`systemctl status ocr-api` and `curl -I http://127.0.0.1:8000/docs` (no SELinux fix needed on Ubuntu).
/ocr_api/docs shows a blank page	`--root-path /ocr_api` is missing in the service file. Add it, then `systemctl daemon-reload && systemctl restart ocr-api`.
Page doesn’t load at all	`ufw status` must be inactive or allow `Nginx HTTP`.

Rocky → Ubuntu command map

Rocky Linux 9	Ubuntu
`dnf install -y epel-release; dnf config-manager --set-enabled crb`	`add-apt-repository -y ppa:deadsnakes/ppa`
`python3.11-devel`	`python3.11-dev python3.11-venv`
`gcc gcc-c++ make`	`build-essential`
`krb5-devel krb5-workstation`	`libkrb5-dev krb5-user`
`tesseract tesseract-langpack-eng/-ara`	`tesseract-ocr tesseract-ocr-eng/-ara`
`mesa-libGL glib2 libgomp`	`libgl1 libglib2.0-0 libgomp1`
`postgresql-server + postgresql-setup --initdb`	`postgresql (no initdb step)`
`redis / redis.service`	`redis-server / redis-server.service`
`systemctl enable --now sshd`	`systemctl enable --now ssh`
`FileZilla as root`	`FileZilla as your own user`
`/etc/nginx/conf.d/ocr_api.conf`	`/etc/nginx/sites-available/ocr_api + link in sites-enabled`
`comment out server { } in nginx.conf`	`rm -f /etc/nginx/sites-enabled/default`
`setsebool -P httpd_can_network_connect 1`	`not needed (no SELinux)`
`firewall-cmd --permanent --add-service=http; --reload`	`ufw allow 'Nginx HTTP'; ufw reload`
`dnf clean all`	`apt clean`
`dnf provides '*/libNAME.so*'`	`apt-file search libNAME.so`
`/usr/share/nginx/html (web root)`	`/var/www/html (web root)`